

Dijkstra's Algorithm to find Shortest Paths from a Source to all

Last Updated : 23 Jul, 2025

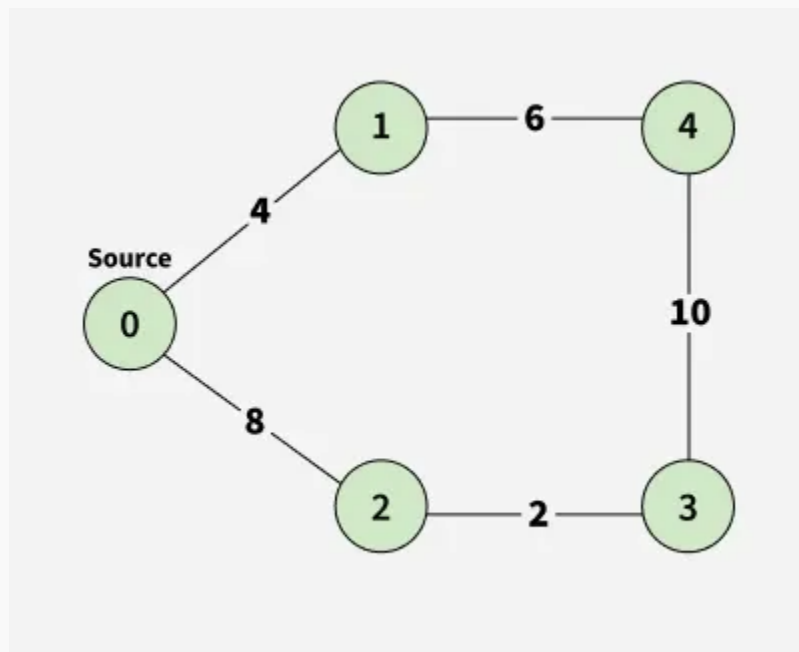


Given a **weighted undirected graph** represented as an **edge** list and a source vertex **src**, find the shortest path distances from the source vertex to all other vertices in the graph. The graph contains **V** vertices, numbered from 0 to $V - 1$.

Note: The given graph does not contain any negative edge.

Examples:

Input: $src = 0$, $V = 5$, $edges[][] = [[0, 1, 4], [0, 2, 8], [1, 4, 6], [2, 3, 2], [3, 4, 10]]$



Graph with 5 node

Output: 0 4 8 10 10

Explanation: Shortest Paths:

0 to 1 = 4. $0 \rightarrow 1$

0 to 2 = 8. $0 \rightarrow 2$

0 to 3 = 10. $0 \rightarrow 2 \rightarrow 3$

0 to 4 = 10. $0 \rightarrow 1 \rightarrow 4$

Dijkstra's Algorithm using Min Heap - $O(E \cdot \log V)$ Time and $O(V)$ Space

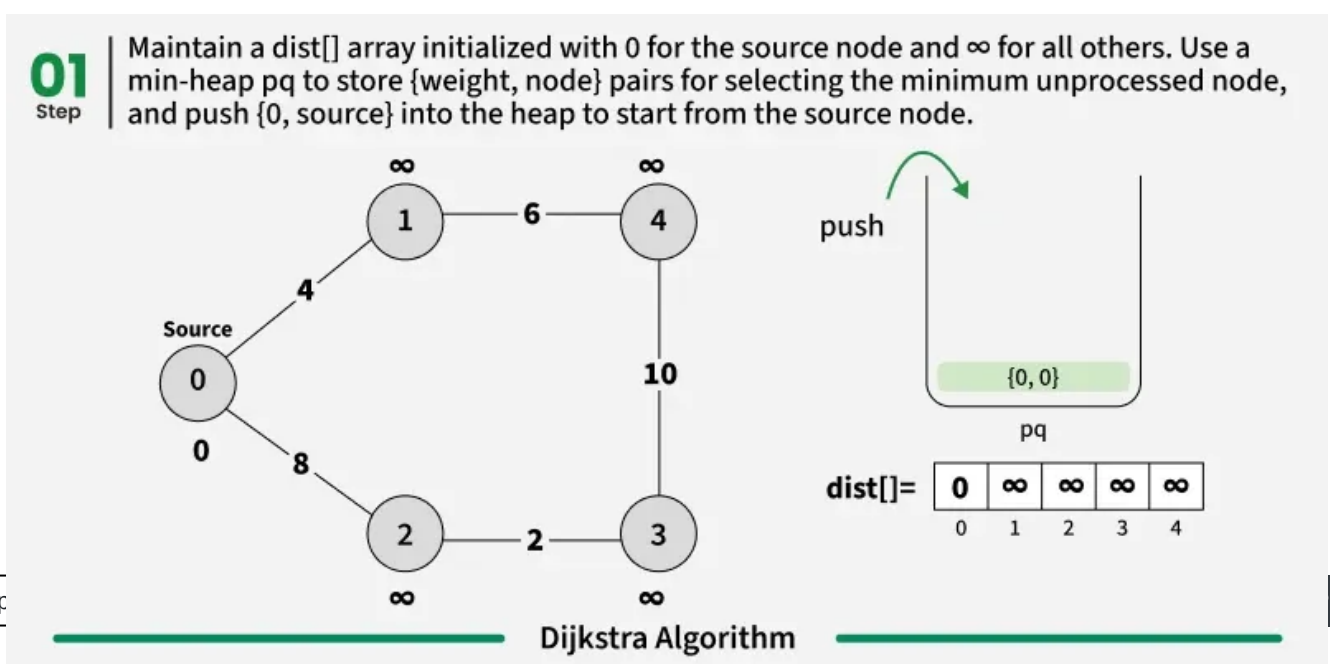
In Dijkstra's Algorithm, the goal is to find the shortest distance from a given source node to all other nodes in the graph. As the source node is the starting point, its distance is initialized to zero. From there, we iteratively pick the unprocessed node with the minimum distance from the source, this is where a min-heap (priority queue) or a set is typically used for efficiency. For each picked node u , we update the distance to its neighbors v using the formula: $\text{dist}[v] = \text{dist}[u] + \text{weight}[u][v]$, but only if this new path offers a shorter distance than the current known one. This process continues until all nodes have been processed.

Step-by-Step Implementation

1. Set $\text{dist}[\text{source}] = 0$ and all other distances as infinity.
2. Push the source node into the min heap as a pair $\langle \text{distance}, \text{node} \rangle \rightarrow \text{i.e., } \langle 0, \text{source} \rangle$.
3. Pop the top element (node with the smallest distance) from the min heap.
 1. For each adjacent neighbor of the current node:
 2. Calculate the distance using the formula:

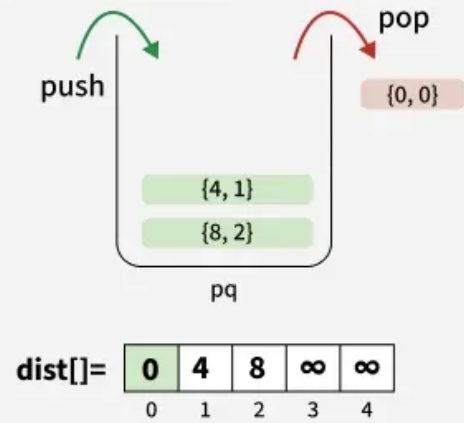
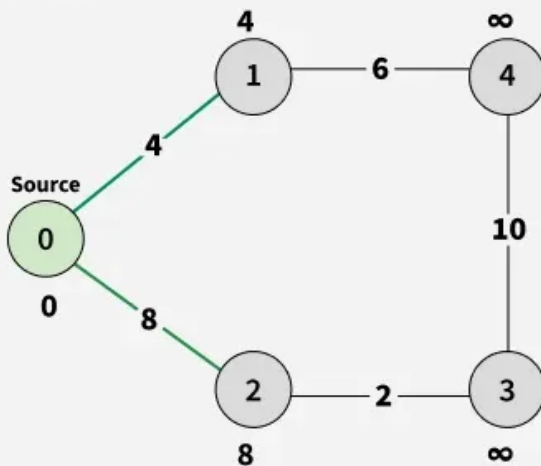
$$\text{dist}[v] = \text{dist}[u] + \text{weight}[u][v]$$
 If this new distance is shorter than the current $\text{dist}[v]$, update it.
 Push the updated pair $\langle \text{dist}[v], v \rangle$ into the min heap
4. **Repeat** step 3 until the min heap is empty.
5. Return the distance array, which holds the shortest distance from the source to all nodes.

Illustration:

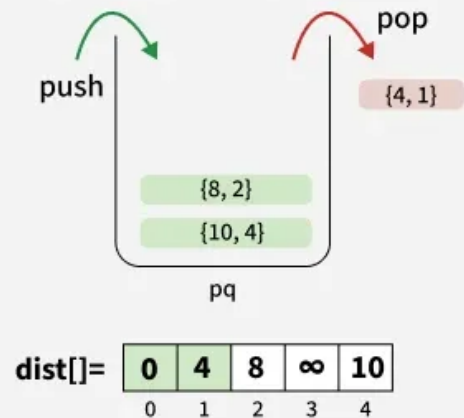
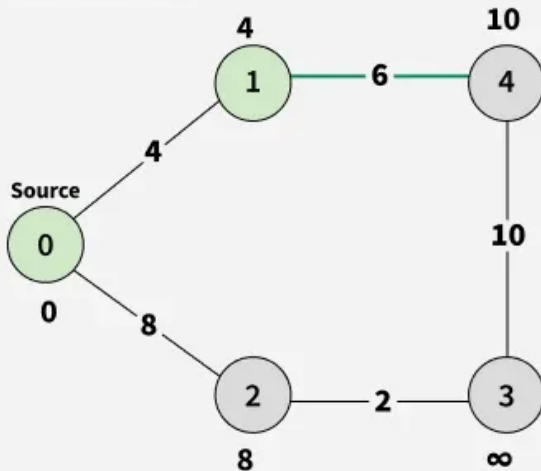


02
Step

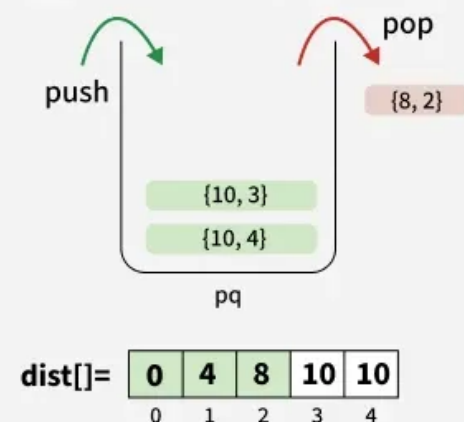
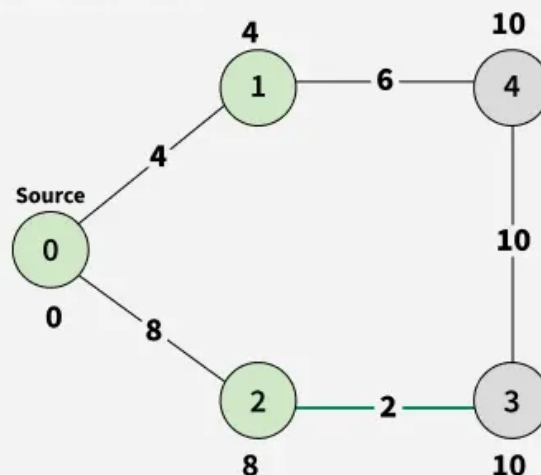
Pop the top element from the heap (e.g., $\{0, 0\}$ where $u = 0$). For each adjacent node v of u , if $\text{dist}[v] > \text{dist}[u] + \text{weight}$, update $\text{dist}[v]$ to $\text{dist}[u] + \text{weight}$ and push $\{\text{dist}[v], v\}$ into the heap

**Dijkstra Algorithm****03**
Step

Pop the top element from the heap (e.g., $\{4, 1\}$ where $u = 1$). For each adjacent node v of u , if $\text{dist}[v] > \text{dist}[u] + \text{weight}$, update $\text{dist}[v]$ to $\text{dist}[u] + \text{weight}$ and push $\{\text{dist}[v], v\}$ into the heap

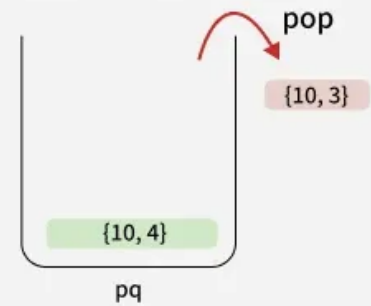
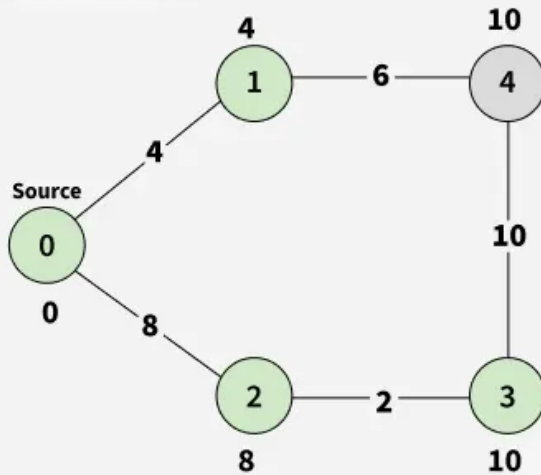
**Dijkstra Algorithm****04**
Step

Pop the top element from the heap (e.g., $\{8, 2\}$ where $u = 2$). For each adjacent node v of u , if $\text{dist}[v] > \text{dist}[u] + \text{weight}$, update $\text{dist}[v]$ to $\text{dist}[u] + \text{weight}$ and push $\{\text{dist}[v], v\}$ into the heap

**Dijkstra Algorithm**

05
Step

Pop the top element from the heap (e.g., $\{10, 3\}$ where $u = 3$). For each adjacent node v of u , if $\text{dist}[v] > \text{dist}[u] + \text{weight}$, update $\text{dist}[v]$ to $\text{dist}[u] + \text{weight}$ and push $\{\text{dist}[v], v\}$ into the heap



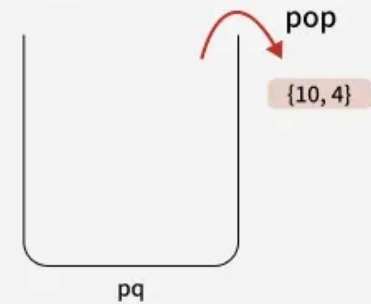
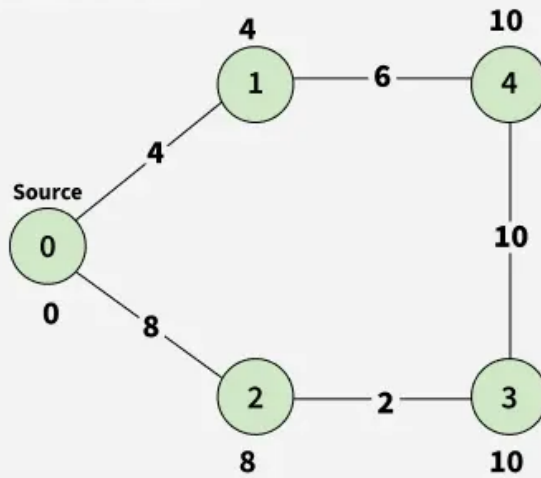
dist[] =

0	4	8	10	10
0	1	2	3	4

Dijkstra Algorithm

06
Step

Pop the top element from the heap (e.g., $\{10, 4\}$ where $u = 4$). For each adjacent node v of u , if $\text{dist}[v] > \text{dist}[u] + \text{weight}$, update $\text{dist}[v]$ to $\text{dist}[u] + \text{weight}$ and push $\{\text{dist}[v], v\}$ into the heap



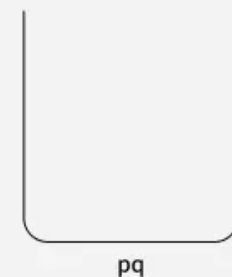
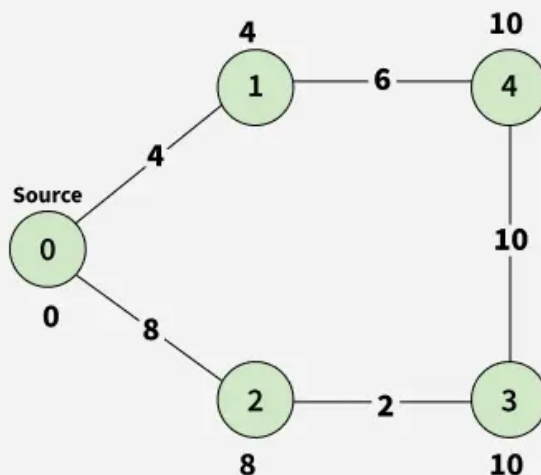
dist[] =

0	4	8	10	10
0	1	2	3	4

Dijkstra Algorithm

07
Step

All nodes are processed and the min-heap is empty, We now have the final shortest distances from the source node 0.



dist[] =

0	4	8	10	10
0	1	2	3	4

Dijkstra Algorithm

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

// Function to construct adjacency
vector<vector<vector<int>>> constructAdj(vector<vector<int>>
                                     &edges, int V) {

    // adj[u] = list of {v, wt}
    vector<vector<vector<int>>> adj(V);

    for (const auto &edge : edges) {
        int u = edge[0];
        int v = edge[1];
        int wt = edge[2];
        adj[u].push_back({v, wt});
        adj[v].push_back({u, wt});
    }

    return adj;
}
```

C++

Java

Python

C#

JavaScript

```
// Returns shortest distances from src to all other vertices
vector<int> dijkstra(int V, vector<vector<int>>> &edges, int src){

    // Create adjacency list
    vector<vector<vector<int>>> adj = constructAdj(edges, V);

    // Create a priority queue to store vertices that
    // are being preprocessed.
    priority_queue<vector<int>, vector<vector<int>>,
                    greater<vector<int>>> pq;

    // Create a vector for distances and initialize all
    // distances as infinite
    vector<int> dist(V, INT_MAX);

    // Insert source itself in priority queue and initialize
    // its distance as 0.
    pq.push({0, src});
    dist[src] = 0;

    // Looping till priority queue becomes empty (or all
    // distances are not finalized)
    while (!pq.empty()){

        // The first vertex in pair is the minimum distance
        // vertex, extract it from priority queue.
        int u = pq.top()[1];
        pq.pop();

        // Get all adjacent of u.
        for (auto x : adj[u]){

            // Get vertex label and weight of current
            // adjacent of u.
```

```

        int v = x[0];
        int weight = x[1];

        // If there is shorter path to v through u.
        if (dist[v] > dist[u] + weight)
        {
            // Updating distance of v
            dist[v] = dist[u] + weight;
            pq.push({dist[v], v});
        }
    }
}

return dist;
}

```

// Driver program to test methods of graph class

```

int main(){
    int V = 5;
    int src = 0;

    // edge list format: {u, v, weight}
    vector<vector<int>> edges = {{0, 1, 4}, {0, 2, 8}, {1, 4, 6},
                                {2, 3, 2}, {3, 4, 10}};

    vector<int> result = dijkstra(V, edges, src);

    // Print shortest distances in one line
    for (int dist : result)
        cout << dist << " ";

    return 0;
}

```

Output

```
0 4 8 10 10
```

Time Complexity: $O(E \cdot \log V)$, Where E is the number of edges and V is the number of vertices.

Auxiliary Space: $O(V)$, Where V is the number of vertices, We do not count the adjacency list in auxiliary space as it is necessary for representing the input graph.

Problems based on Shortest Path

- [Shortest Path in Directed Acyclic Graph](#)
- [Shortest path with one curved edge in an undirected Graph](#)
- [Minimum Cost Path](#)
- [Path with smallest difference between consecutive cells](#)
- [Print negative weight cycle in a Directed Graph](#)
- [1st to Kth shortest path lengths in given Graph](#)
- [Shortest path in a Binary Maze](#)
- [Minimum steps to reach target by a Knight](#)
- [Number of ways to reach at destination in shortest time](#)
- [Snake and Ladder Problem](#)

- [Word Ladder](#)

Dijkstra's Shortest Path Algorithm

Visit Course ↗

Comment

K

kartik

+ Follow

👍

✎

Article Tags :

- Graph
- Greedy
- DSA
- Amazon
- +8 More

Explore

DSA Fundamentals	▼
Data Structures	▼
Algorithms	▼
Advanced	▼
Interview Preparation	▼
Practice Problem	▼

Do Not Sell or Share My Personal Information